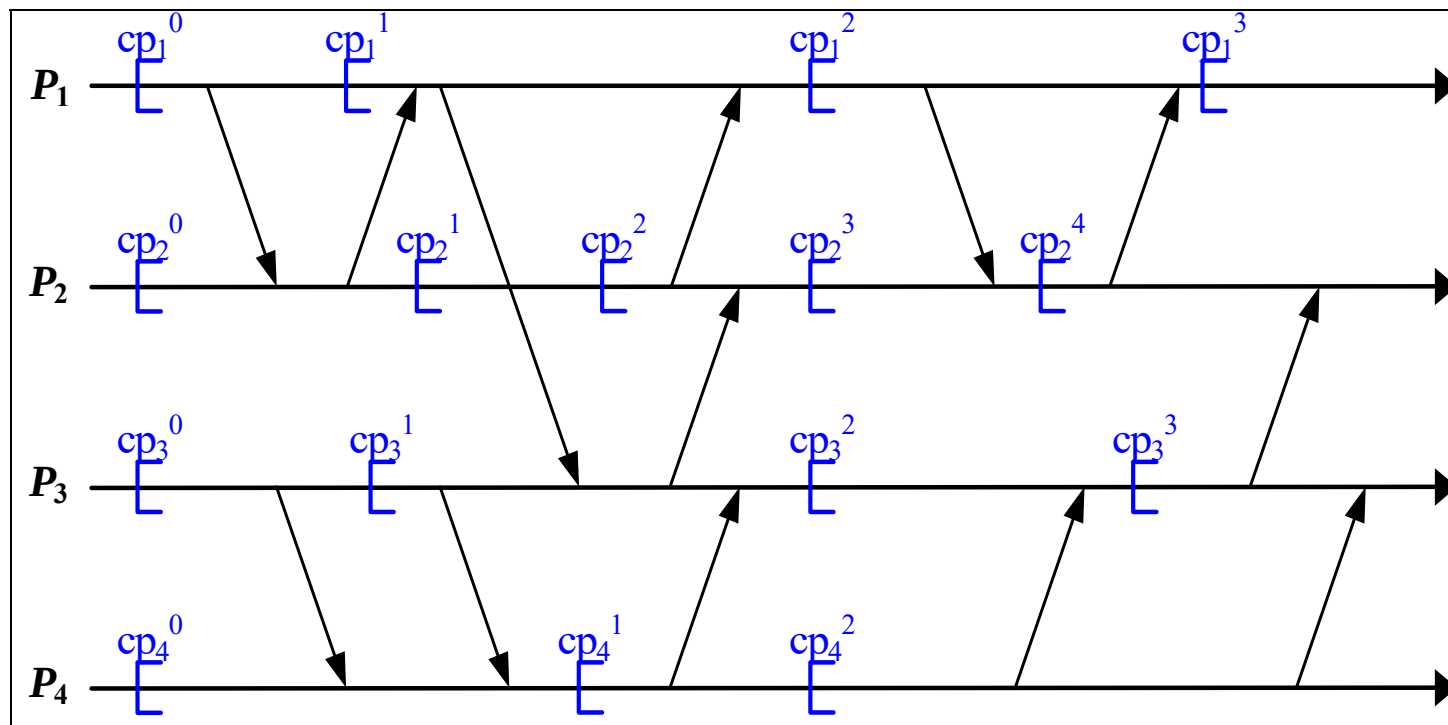




HYBRID CHECKPOINTING

- = asynchronous checkpointing (failure free performance)
- + occasional synchronization gets an RL (to finally avoid domino danger)





HYBRID CHECKPOINTING

Typical solution:

- independent checkpointing + message logging (optimistic)
- occasional global checkpoint coordination
- e.g. synchronization and access to stable storage only on output commit
- ☹ but recovery protocols become quite complex



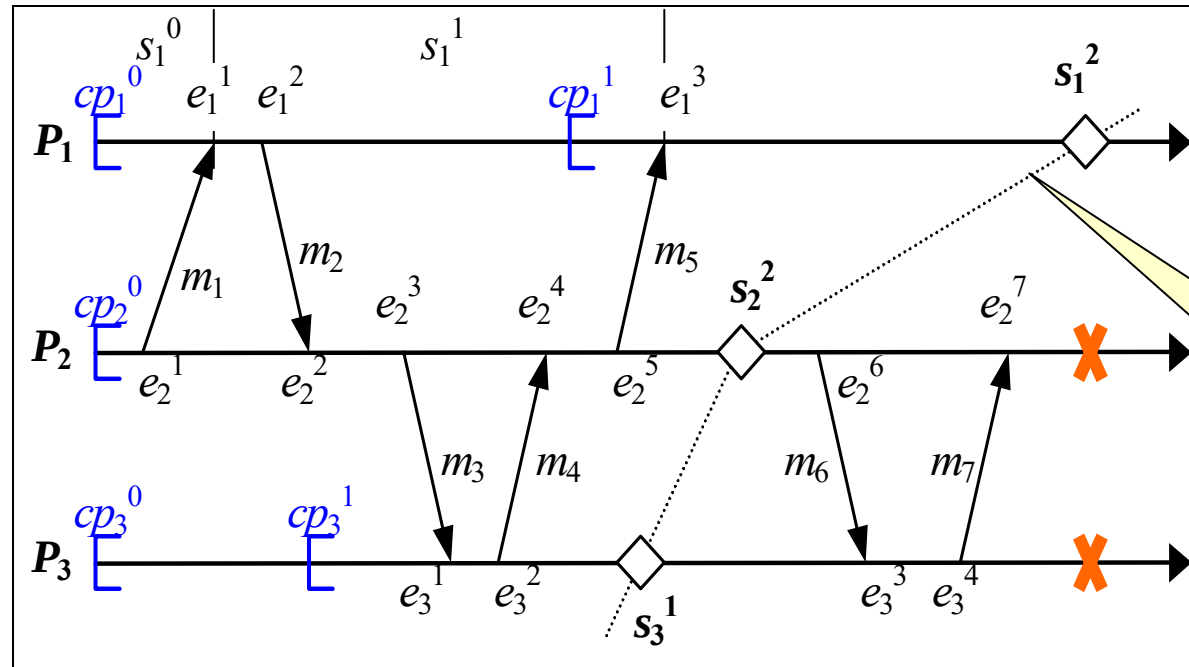
Causal logging

- isolates a process from the effects of other process failures
- limits the rollback of any failed process to the most recent checkpoint on stable storage

Basic invariant of causal logging:

- information about each event that causally precedes^{*)} the state of a process is either
 - fully logged
 - or
 - available locally to the process

^{*)} according to the *happened-before* relation ($\rightarrow$)



maximum
recoverable
state line

While messages m_6 and m_7 may be lost upon the failure, process P_1 at state s_1^2 would have information about all nondeterministic events that precede its state in causal order. These events consist of the receipts of messages m_1 , m_2 , m_3 , m_4 and m_5 . Thus, controller C_1 will be able to guide the recovery of P_2 and P_3 since it knows the order in which P_2 should replay messages m_2 and m_4 to reach state s_2^2 , and the order in which P_3 should replay m_3 to reach state s_3^1 . Such messages can be replayed from the sender log of P_1 or will be regenerated during the recovery of P_2 and P_3 .



Manetho algorithm [1]

System model:

- fail-recovery process failures
- unreliable reordering channels
- computation is piecewise deterministic (sequence of deterministic state intervals, each started by a nondeterministic event)
- all nondeterministic events (recv, internal) are logged optimistically
- output commit

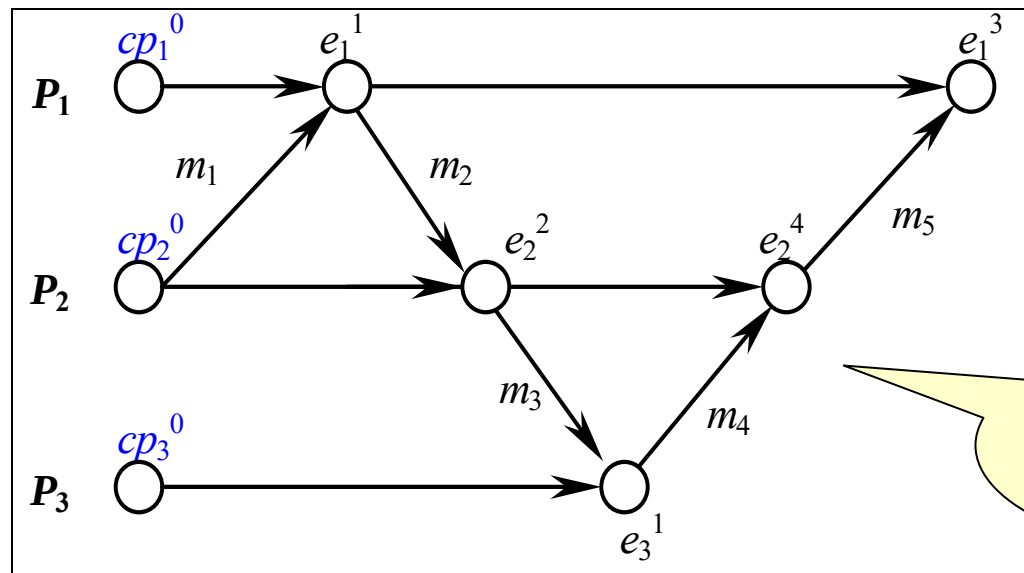
General idea:

- causal logging with the use of an antecedence graph

Antecedence graph (graf poprzedzania przyczynowego)

- provides every process with a complete history of the nondeterministic events that causally affected its state

Def. The *antecedence graph* $AG(s_i^k)$ of a state s_i^k is composed of the node representing state interval s_i^{k*} , nodes representing nondeterministic events preceding s_i^k and edges corresponding to the *happened-before* relation.



antecedence graph
on P_1 at state s_1^2
(see slide 4)

^{*}) actually, a nondeterministic event beginning the piecewise deterministic interval



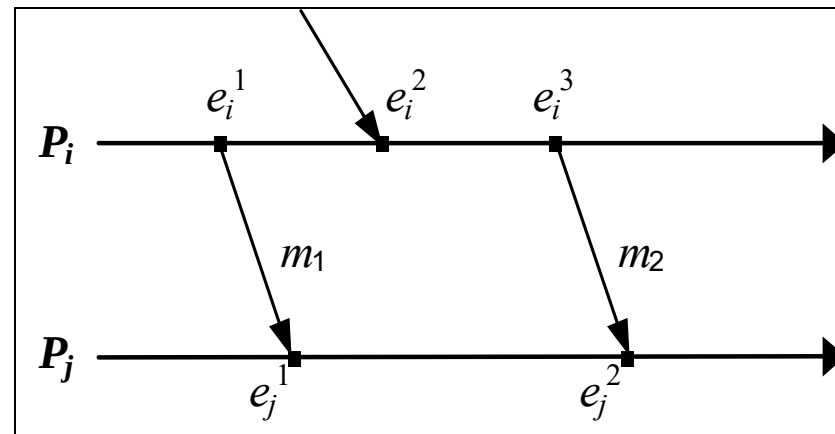
Piggybacking overhead

- AG is piggybacked on each application message sent
- however, carrying the entire AG on each message may lead to an unacceptable overhead
- fortunately it is clear, that a new message carries a graph that is a supergraph of the one piggybacked on the previous message sent through the same channel
- practically then, Manetho uses incremental piggybacking ...



Incremental piggybacking

- 1) for any subsequent m_1, m_2 sent from P_i to P_j , m_2 carries only the difference^{*)} $m_2.AG_i$ between $m_1.AG_i$ and current AG_i , i.e. $m_2.AG_i = AG_i - m_1.AG_i$

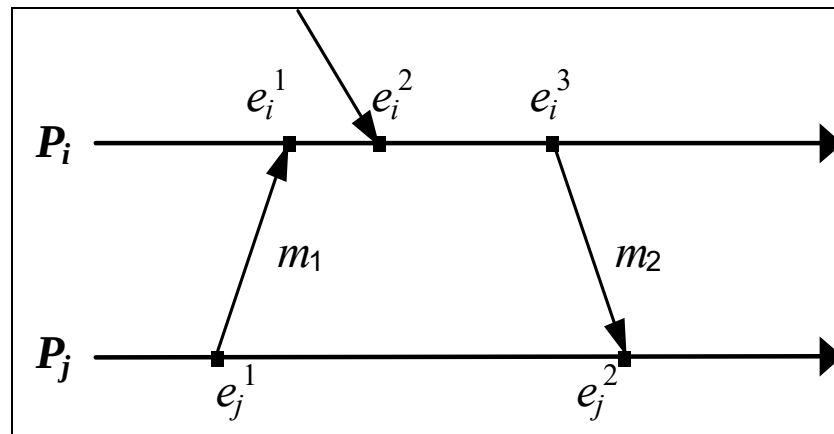


^{*)} uwaga na własności kanałów (zawodne nonFIFO)!



Incremental piggybacking

- 2) if P_i has previously received m_1 from P_j , then C_i can exclude $m_1.AG_j$ from $m_2.AG_i$
(no need to send $m_1.AG_j$ back to C_j), i.e. $m_2.AG_i = AG_i - m_1.AG_j$





QUASI-SYNCHRONOUS CHECKPOINTING

Manivannan-Singhal algorithm [2]

- independent communication-induced checkpointing
- selective pessimistic message logging (receiver side)
- failed process needs to rollback only to its latest checkpoint (asynchronously),
a CP is ensured consistent with the latest local checkpoint of any process → RL
- occasional global checkpoint coordination
- no domino effect

System model:

- unreliable reordering channels
- fail-recovery failure model



General idea:

- independently taken checkpoints are called *basic checkpoints*
- communication-induced triggered by message receptions are called *forced checkpoints*
- the *forced checkpoints* help in advancing the RL

Structures:

- each msg is piggybacked with the checkpoint interval number $ckpt_num_i$ (seq. no. of the latest local checkpoint) of the sender P_i
- value of $next_i$ denotes the seq. no. to be assigned to the next basic checkpoint that C_i will take



General idea (cont.):

- value of $next_i$ ticks every Δt_i time unit
- ideally (but not necessary) all Δt_i ($i=1..N$) are close to each other
- *basic checkpoints* are taken on $x \cdot \Delta t_i$ intervals
- the main purpose of $next_i$ is to keep the interval numbers (of the latest checkpoint) of all the processes as similar as possible

QS checkpointing algorithm for C_i :

```
ckpt_numi := 0
nexti := 1
```

on tick:

```
    nexti := nexti + 1
```

on taking a basic ckpt:

```
    if nexti > ckpt_numi then
        ckpt_numi := nexti
        save new cpi
        cpi.ckpt_num := ckpt_numi
    else
        skip taking the new cpi
```

when sending message m:

```
    m.ckpt_num := ckpt_numi
    send m
```

on reception of message m:

```
    if m.ckpt_num > ckpt_numi then
        ckpt_numi := m.ckpt_num
        save new cpi
        cpi.ckpt_num := ckpt_numi
    deliver the message
```

// ckpt synchro negative

// ckpt synchro positive

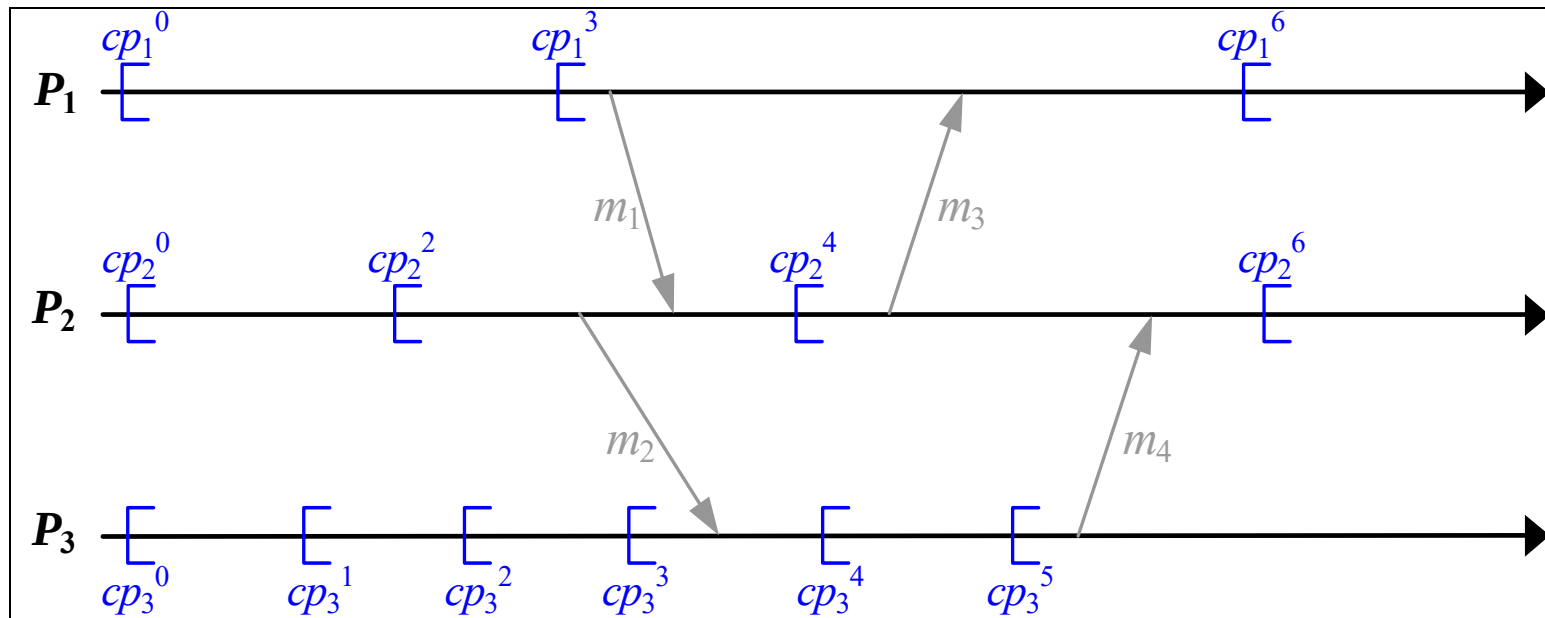
// forced ckpt

C_i has already
taken a forced
checkpoint with
 $ckpt_num_i \geq next_i$



QS checkpointing example:

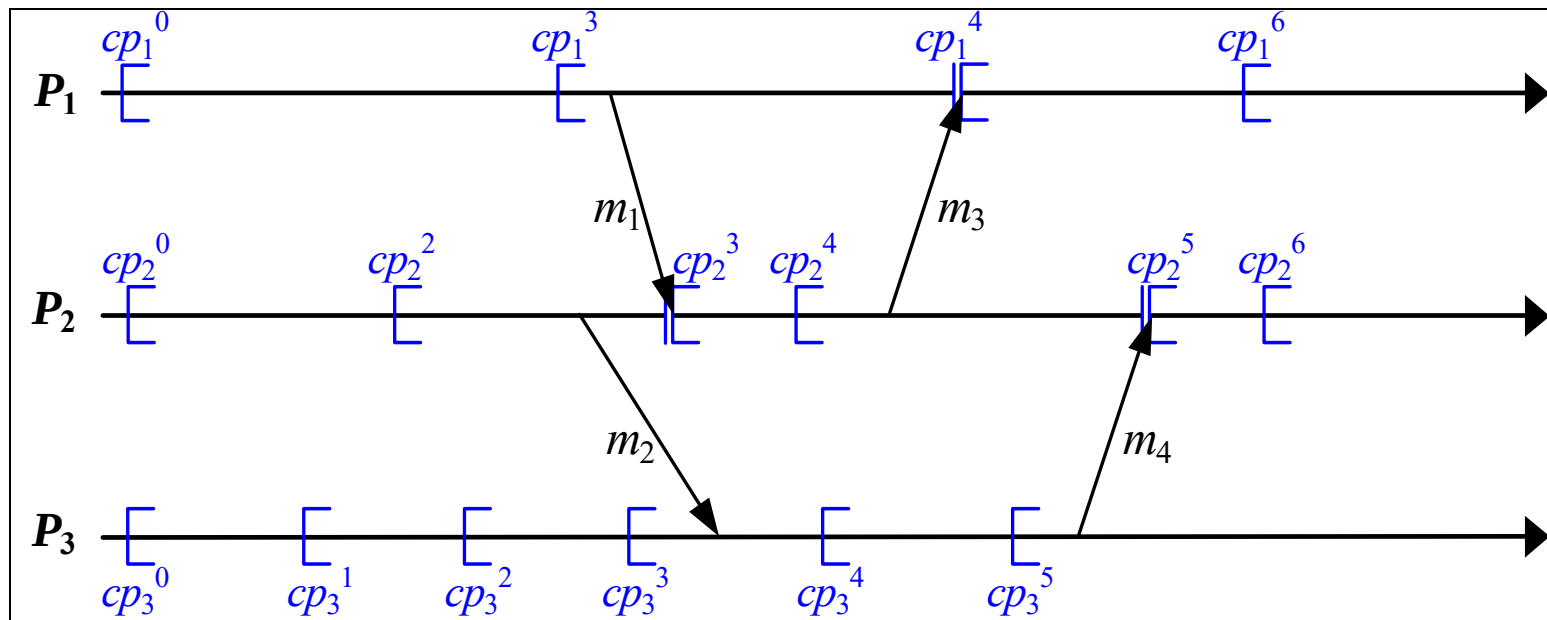
- P_1 takes basic checkpoint every $3 \Delta t_1$
- P_2 takes basic checkpoint every $2 \Delta t_2$
- P_3 takes basic checkpoint every single Δt_3





QS checkpointing example:

- m_1 forces P_2 to take a forced checkpoint cp_2^3 before processing m_1
since $m_1.ckpt_num=3$ and $ckpt_num_2=2$ ($m_1.ckpt_num > ckpt_num_2$)
- m_2 does not force P_3 to take a checkpoint ($m_2.ckpt_num \not> ckpt_num_3$)





Recovery

Assumption:

- if a process fails, no other process fails until all processes are rolled back to RL

basic QS recovery algorithm for C_i :

```
on faili:  
    roll back to the latest  $cp_i$   
    send Rollback( $cp_i.ckpt\_num$ ) to all other  $C_j$   
  
on receiving Rollback( $rp\_num$ ):  
    if  $ckpt\_num_i \geq rp\_num$  then  
        (1)    find the earliest  $cp_i::cp_i.ckpt\_num \geq rp\_num$   
        (2)    roll back to that  $cp_i$   
    else                                     //  $P_i$  does not roll back at all  
        (3)    save new  $cp_i$                   // similar to  $vcp_i$   
                 $cp_i.ckpt\_num := rp\_num$   
                 $ckpt\_num_i := cp_i.ckpt\_num$ 
```



Observation:

- all processes restart from the earliest checkpoint $:: cp_i.ckpt_num \geq rp_num$ (1)
- since they either have been rolled back to (2) or continue from (3) that rp_num

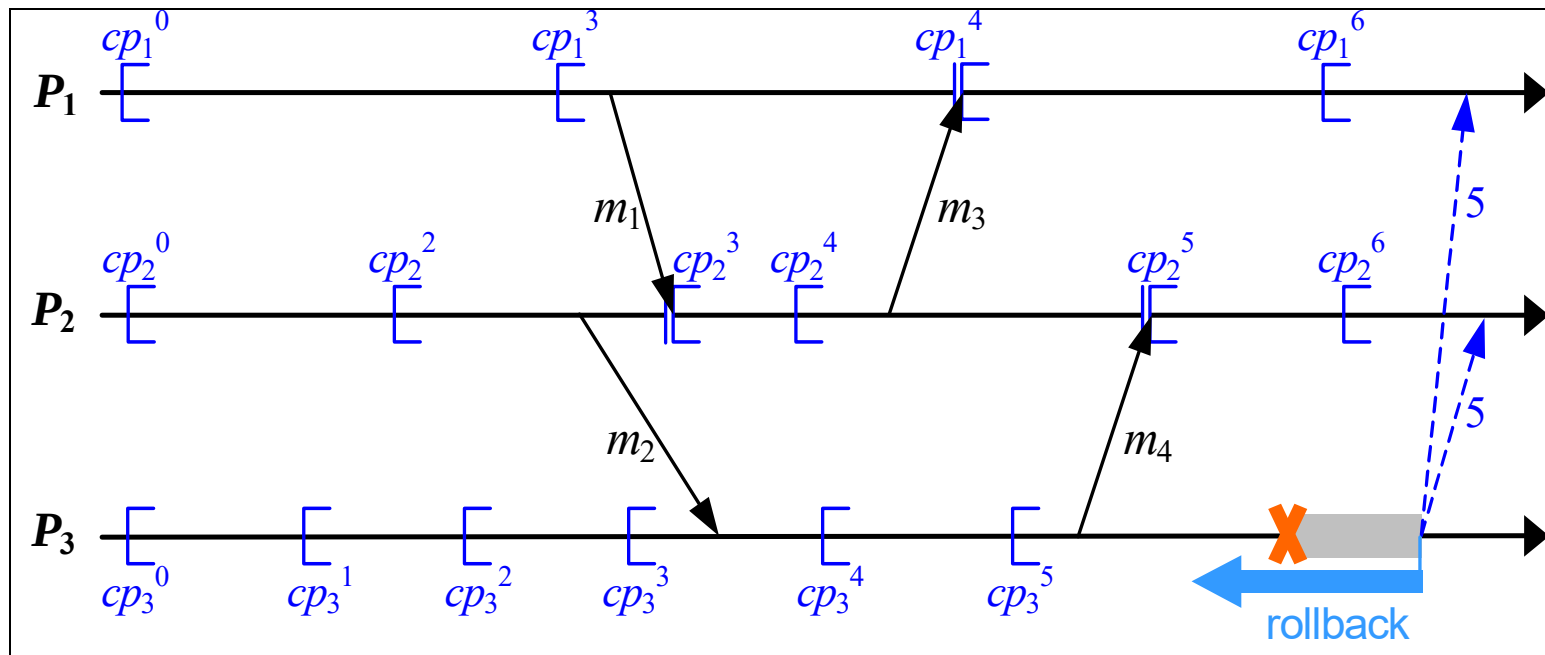
Correctness:

- the checkpoints from which the processes restart form a $CP^\bullet$, i.e. a correct RL



Example 1:

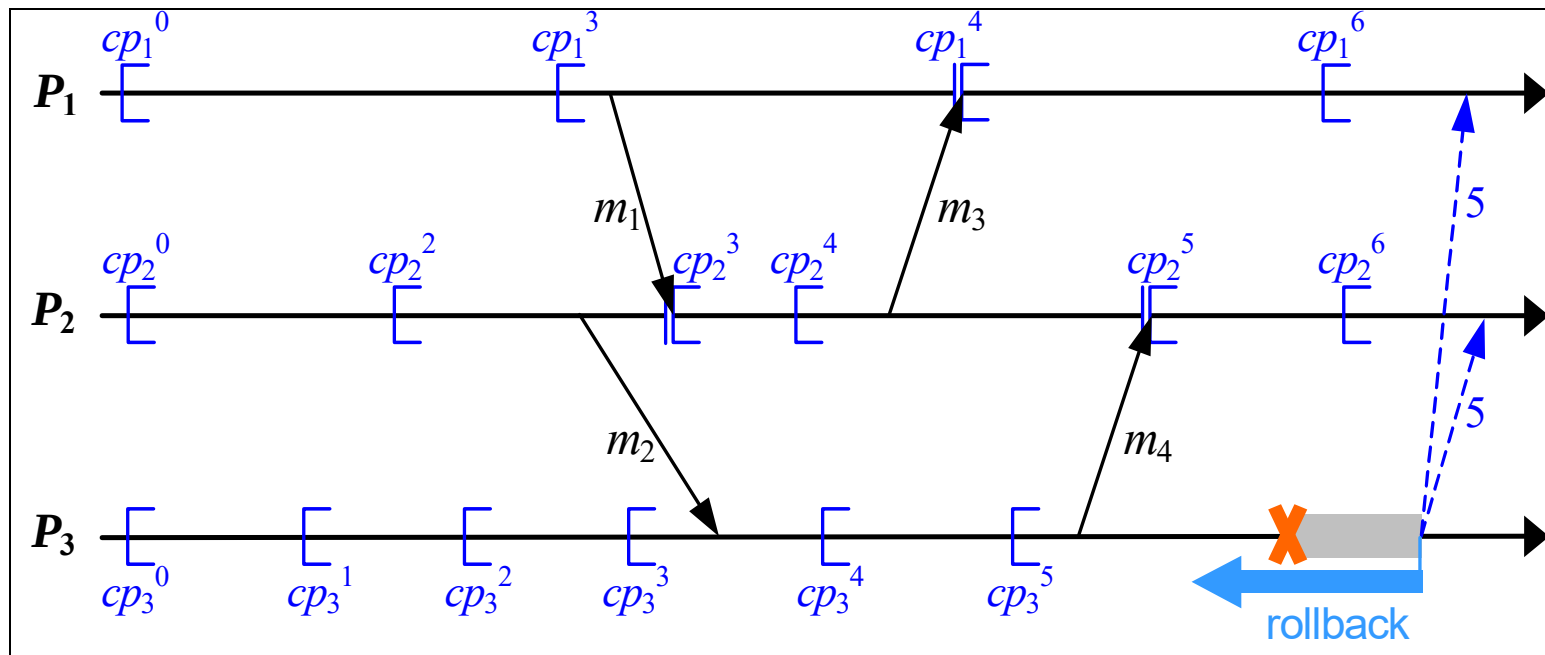
- C_3 will roll back P_3 to cp_3^5 and send *Rollback* (5) to C_1 and C_2
- C_2 will roll back P_2 to cp_2^5 (the earliest $cp_2.ckpt_num \geq 5$)
- C_1 will roll back P_1 to cp_1^4 (the earliest $cp_1.ckpt_num \geq 5$)





Example 1:

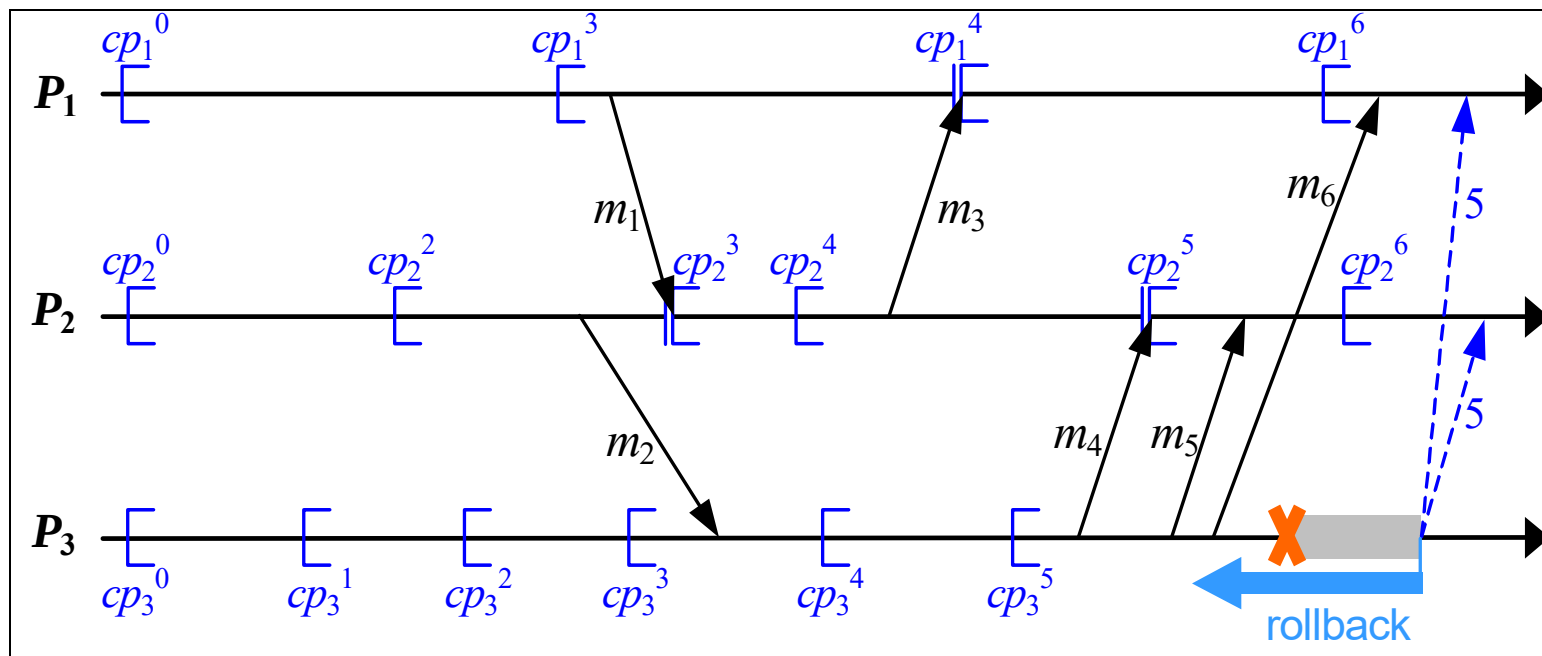
- does C_2 need to roll back P_2 to cp_2^5 ?
- does C_1 need to roll back P_1 to cp_1^6 ?





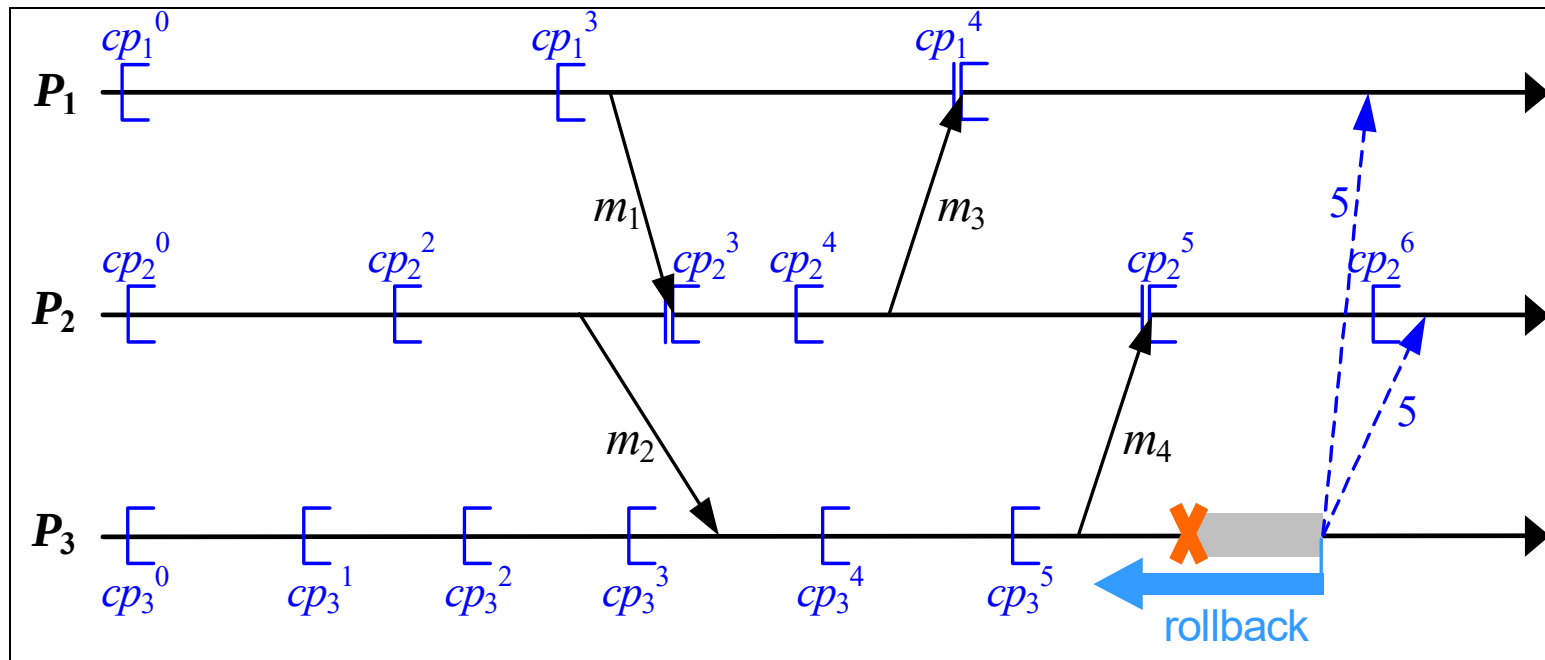
Example 1:

- does C_2 need to roll back P_2 to cp_2^5 ?
- does C_1 need to roll back P_1 to cp_1^6 ?





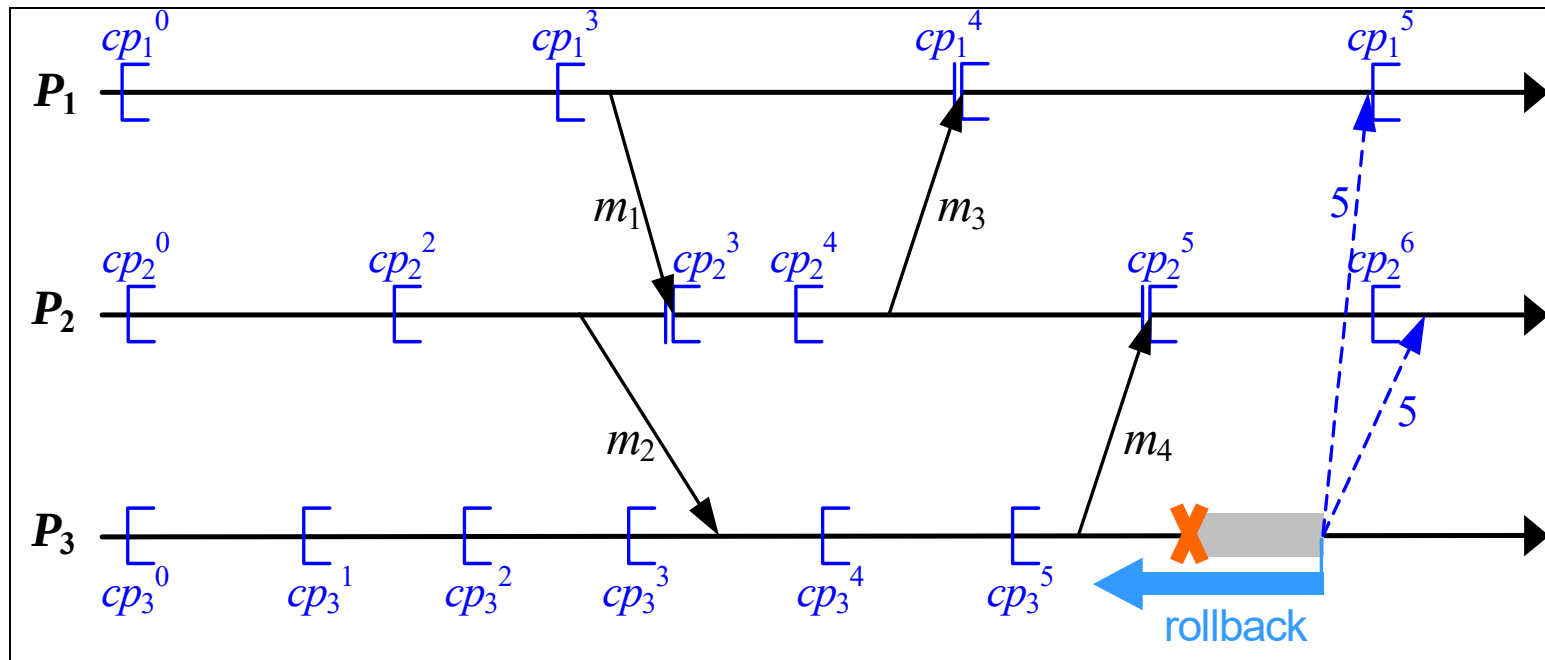
Example 2:



- C_1 will take a new cp_1^5 (since it does not have any $cp_1.ckpt_num \geq 5$)



Example 2:



- C_1 will take a new cp_1^5 (since it does not have any $cp_1.ckpt_num \geq 5$)



Claim C1:

If P_i is rolled back to $cp_i^{x_i}$ upon receiving *Rollback* (rp_num) from C_f then

- a) $x_i \geq rp_num$ but
- b) all checkpoints taken by C_i prior to $cp_i^{x_i}$ have interval numbers less than rp_num

Claim C2:

For any message m sent by P_i : $\overrightarrow{send}_i(P_i, P_j, m) \in cp_i^{x_i} \Leftrightarrow m.ckpt_num < x_i$

Claim C3:

For any message m received by P_i : $\overleftarrow{recv}_i(P_i, P_j, m) \in cp_i^{x_i} \Rightarrow m.ckpt_num < x_i$

Claim C4:

For any i : C_i delivers a received message m to P_i only after it had taken a checkpoint $cp_i^{x_i}$: $x_i \geq m.ckpt_num$

**Theorem 1:**

Assume that upon receiving $Rollback(r)$ from C_f process P_i is rolled back to $cp_i^{x_i}$.

Then the set $CP = \{cp_1^{x_1}, \dots, cp_f^r, \dots, cp_i^{x_i}, \dots, cp_N^{x_N}\}$ is a correct RL.

safety property

Proof:

(P0) from C1.a) we have $\forall_{i=1..N} x_i \geq r$

by contradiction:

(P1) suppose CP is not a consistent CP[•]

...



(P0) from C1.a) we have $\forall_{i=1..N} x_i \geq r$

by contradiction:

(P1) suppose CP is not a consistent $CP^\bullet$, i.e. $\exists_m \text{recv}_i(\mathbf{P}_i, \mathbf{P}_j, \mathbf{m}) \in cp_i^{x_i} \wedge \text{send}_j(\mathbf{P}_j, \mathbf{P}_i, \mathbf{m}) \notin cp_j^{x_j}$

(P2) so $\mathbf{m}.ckpt_num \geq x_j$ from C2

(P3) and $\mathbf{m}.ckpt_num < x_i$ from C3

(P4) from (P0), (P2) and (P3) we get: $r \leq x_j \leq \mathbf{m}.ckpt_num < x_i$

(P5) from C4: $\mathbf{P}_i$ must have processed $\mathbf{m}$ only after a cp_i^x had been taken $\therefore x \geq \mathbf{m}.ckpt_num$

(P6) since $r \leq \mathbf{m}.ckpt_num$ (P4) and $x \geq \mathbf{m}.ckpt_num$ (P5) $\Rightarrow r \leq \mathbf{m}.ckpt_num \leq x \Rightarrow x \geq r$

(P7) since $\text{recv}_i(\mathbf{P}_i, \mathbf{P}_j, \mathbf{m}) \in cp_i^{x_i}$ then $\mathbf{m}$ must have been processed by $\mathbf{P}_i$ before $cp_i^{x_i}$

(P8) hence this cp_i^x such that $x \geq r$ (P6) has been taken before $cp_i^{x_i}$

(P9) but from C1.b): all cp_i^x prior to $cp_i^{x_i}$ have interval numbers x less than r

hence the assumption that CP is not consistent (P1) was wrong

thus CP is $CP^\bullet$ and constitutes a RL





Garbage collection

extension of the basic QS recovery algorithm for C_i :

```
on faili:  
    roll back to the latest  $cp_i$   
    send Rollback( $cp_i.ckpt\_num$ ) to all other  $C_j$   
  
on receiving Rollback( $rp\_num$ ):  
    if  $ckpt\_num_i \geq rp\_num$  then  
        find the earliest  $cp_i::cp_i.ckpt\_num \geq rp\_num$   
        roll back to that  $cp_i$   
        discard all the checkpoints beyond  $cp_i$   
    else //  $P_i$  does not roll back at all  
        save new  $cp_i$  // similar to  $vcp_i$   
         $cp_i.ckpt\_num := rp\_num$   
         $ckpt\_num_i := cp_i.ckpt\_num$ 
```



Recovery – the complete QS recovery algorithm

- from **Theorem 1** all processes roll back to CP^*
- however, restoring a consistent state involves also proper handling of messages received/sent before and after the rollback
- complete QS recovery uses selective pessimistic message logging on the receiver side
- for only those messages that would probably need to be replayed if the process rolls back to the checkpoint that precedes the message reception (reduced logging overhead)
- message ***m*** received after cp_i is logged into $cp_i.msg_log$

Assumption:

- messages received while a process is down are buffered (will be delivered after recovery)



Structures:

If P_i fails, C_i initiates recovery with a new

- incarnation no. — inc_num_i (initially 0)
- recovery line no. — RL_num_i (initially 0)

which both uniquely determine that recovery

- each m is piggybacked with current inc_num_i ($m.inc_num$) and RL_num_i ($m.RL_num$)

When P_i fails:

- C_i rolls P_i back to $cp_i^{x_i}$
- increments inc_num_i
- sets $RL_num_i := cp_i^{x_i}.ckpt_num$ ($=x_i$)
- and sends $Rollback(inc_num_i, RL_num_i)$ to all other C_j



On receipt of $Rollback(inc_num, RL_num)$ the receiver C_i does:

- $inc_num_i := inc_num$
- $RL_num_i := RL_num$
- and rolls back to its earliest checkpoint $cp_i^{x_i} :: cp_i^{x_i}.ckpt_num \geq RL_num$
- if C_i does not have such cp_i (all checkpoints of P_i have seq. no. less than RL_num), it does not roll back but takes a new $cp_i^{x_i} :: cp_i^{x_i}.ckpt_num := RL_num$
- (else) C_i combines the message logs from checkpoints that succeed $cp_i^{x_i}$
- and assigns that as $cp_i^{x_i}.msg_log$ (so these are all the messages logged after $cp_i^{x_i}$)

Observation:

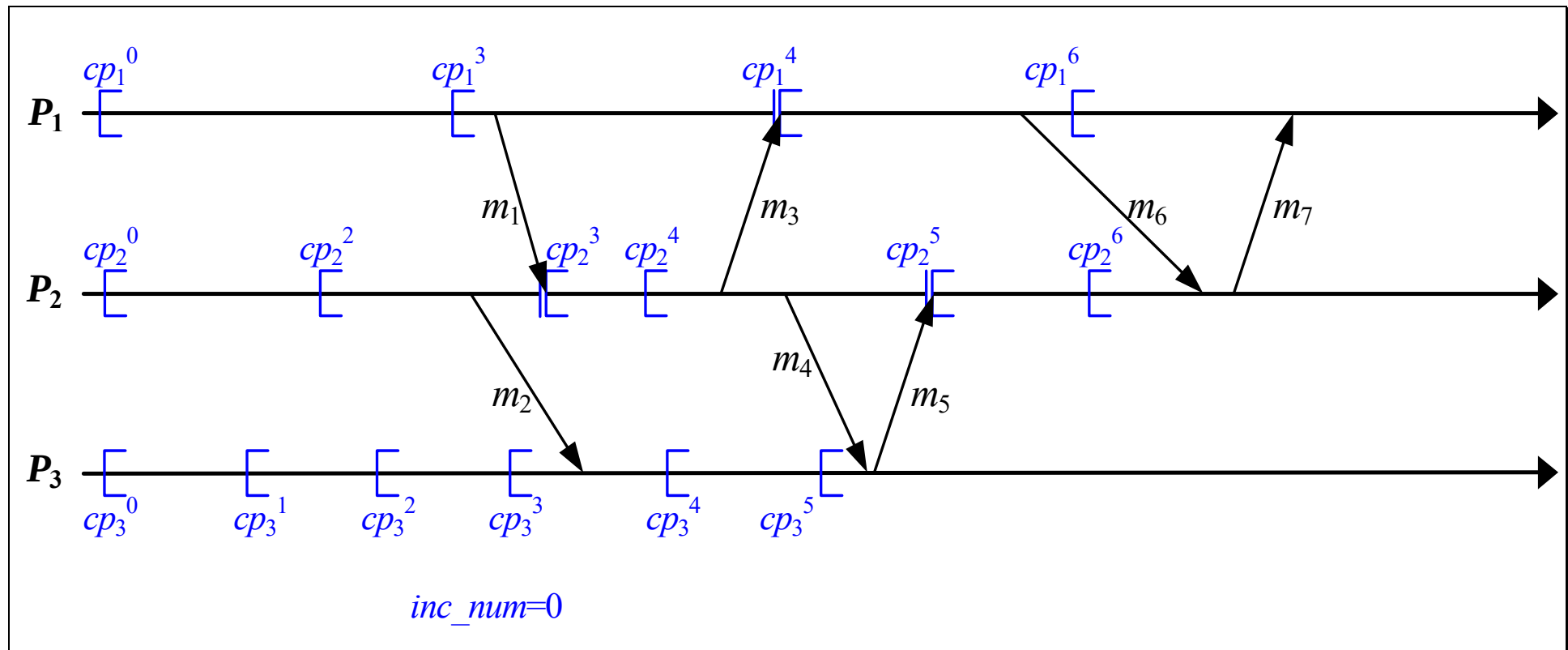
All processes restart from the earliest checkpoint $:: cp_i.ckpt_num \geq RL_num$

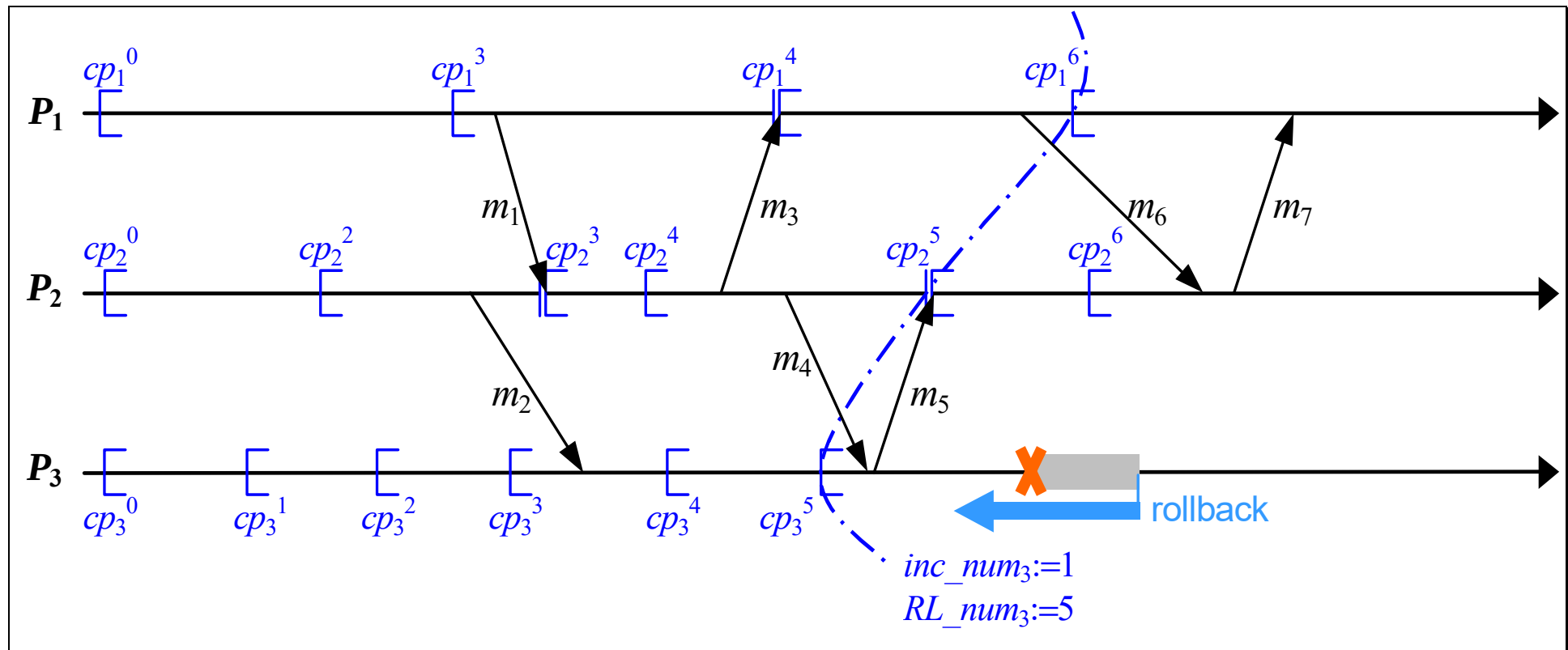
(The local checkpoints form a $CP^\bullet$ — from **Theorem 1**)



Recovery (cont.):

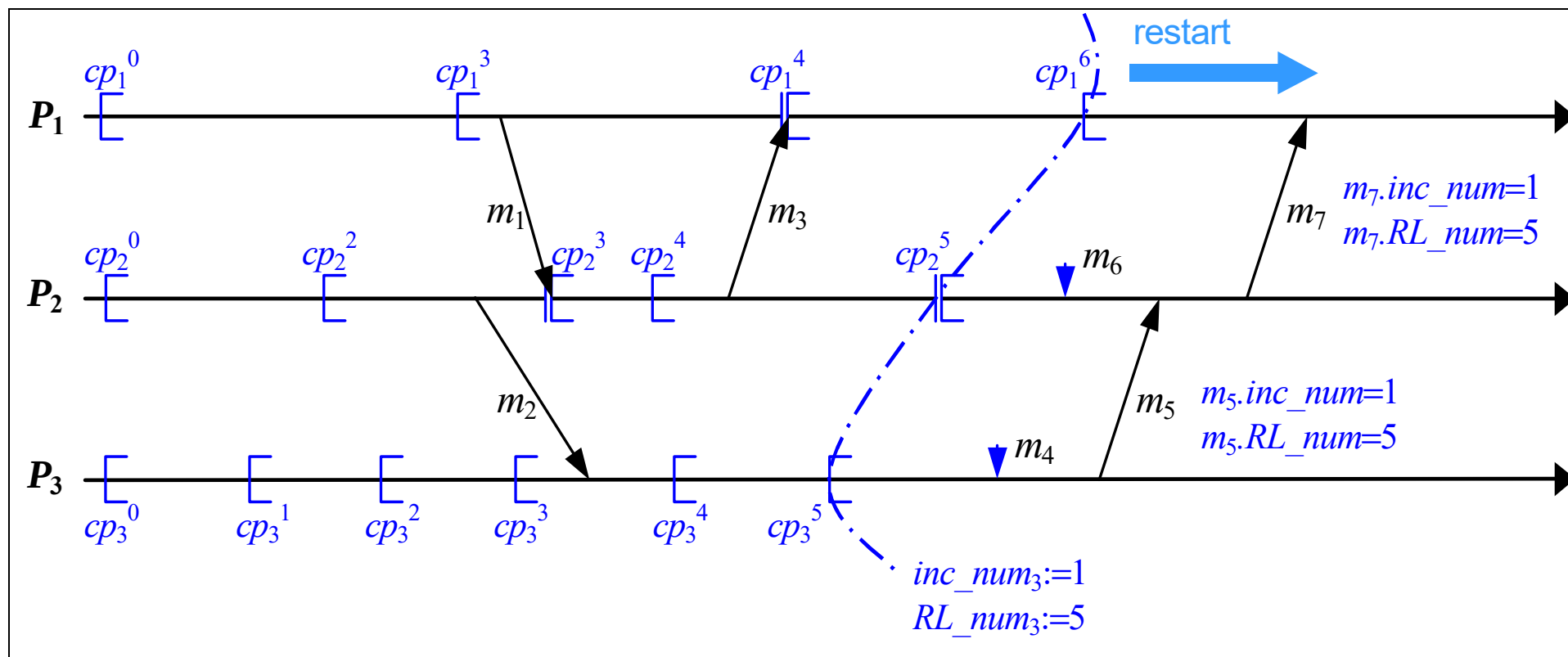
- when C_i restores $cp_i^{x_i}$ it replays from $cp_i^{x_i}.msg_log$ only those messages whose *send* will not be undone
- i.e. messages originated to the left of the current RL and delivered to the right of it
- formally:
after rollback to $cp_i^{x_i}$ a msg m is replayed from log iff
 m was previously received after $cp_i^{x_i}$ and $m.ckpt_num < RL_num_i$
- that takes care of lost messages (avoiding duplication)





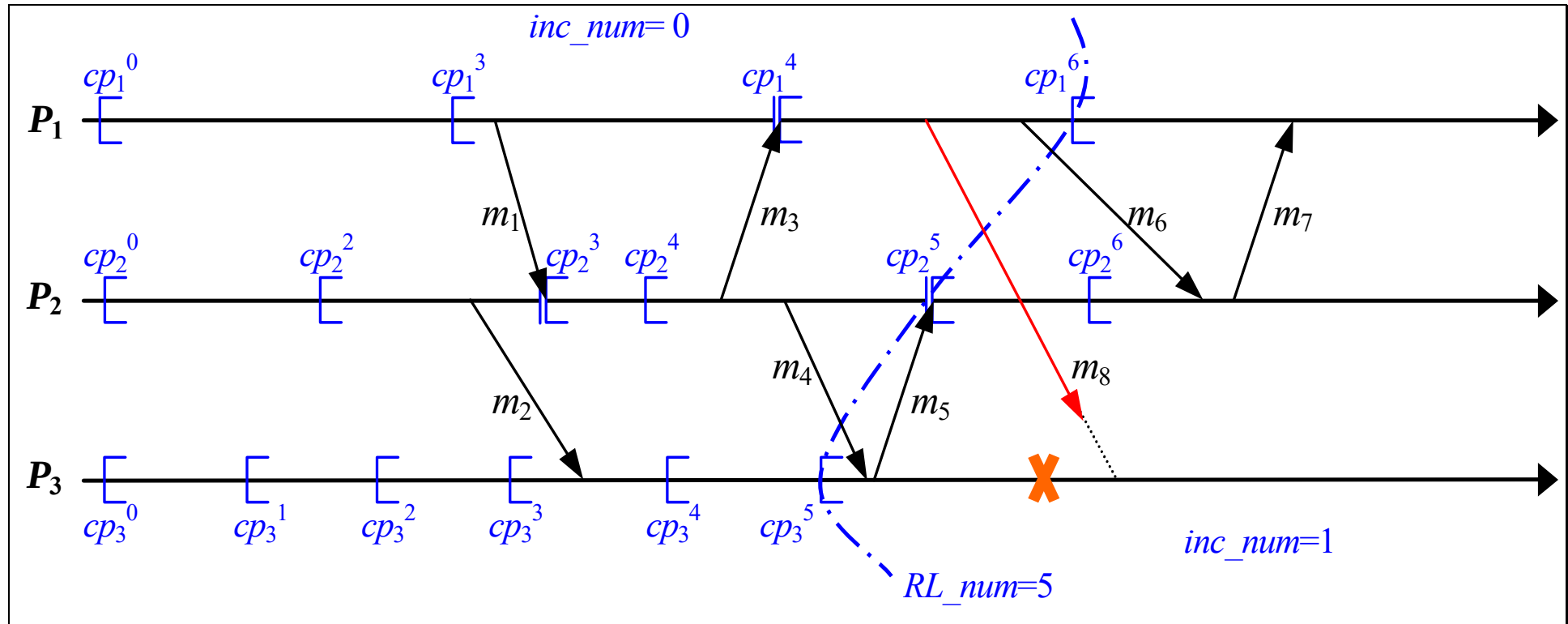
- m_4 and m_6 will be replayed (since sending them has not been undone) ← **lost messages**
- m_5 and m_7 will not be replayed (as their send events were undone) ← **no duplication**

Re-execution:



- m_5 and m_7 are resent in the new incarnation

Delayed messages:

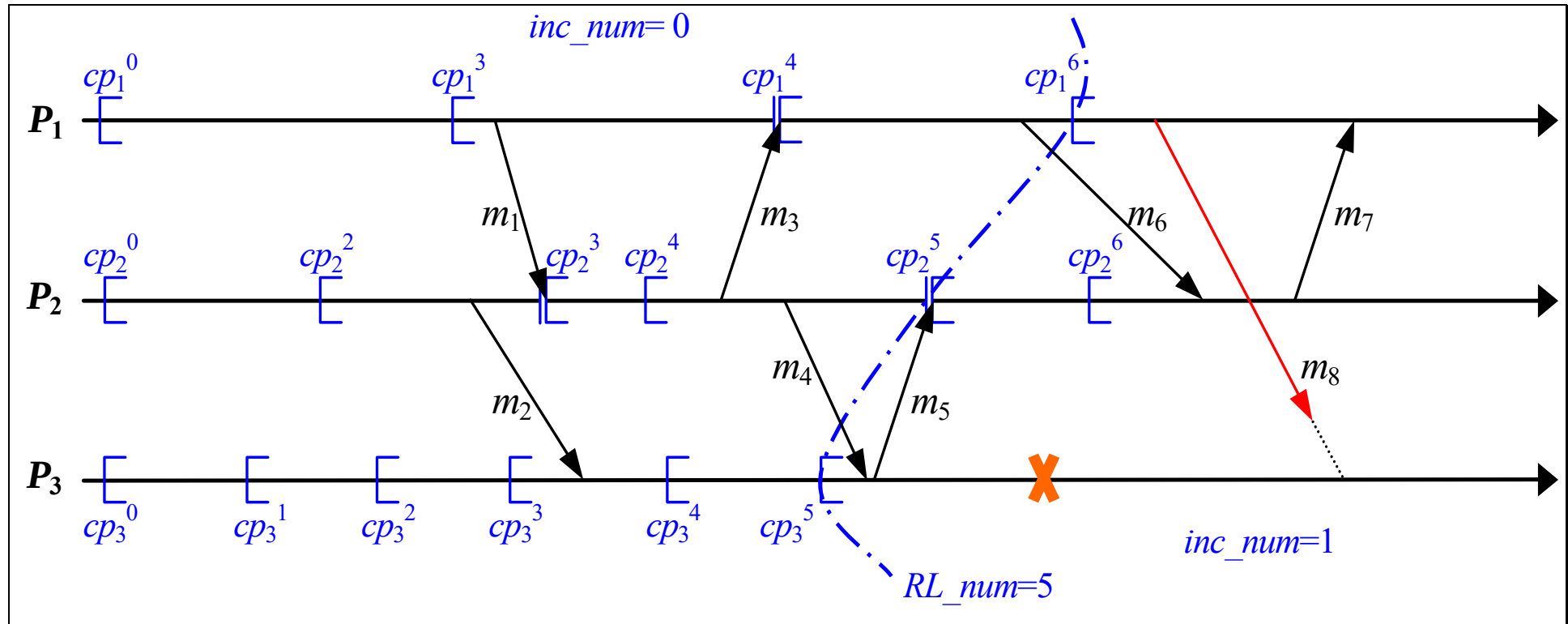


- m_8 is **delayed** (will be received after the recovery)

formally:

- m received by P_i is **delayed** iff $m.inc_num < inc_num_i$ and $m.ckpt_num < RL_num_i$

Duplicated messages:



- m_8 is **duplicated** (will be received after the recovery but also resent in the next incarnation)

formally:

- m received by P_i is **duplicated** iff $m.inc_num < inc_num_i$ and $m.ckpt_num \geq RL_num_i$



Handling messages

On $recv_i(P_i, P_j, m)$ by C_i :

(Case 1) m was sent in previous incarnation ($m.inc_num < inc_num_i$)

- the sender P_j was not aware of the recovery at the time of sending m
- the sender P_j will rollback prior to the send iff $m.ckpt_num \geq RL_num_i$
- thus receiver P_i should process m iff $m.ckpt_num < RL_num_i \leftarrow$ **delayed**
- otherwise C_i should discard m since its send will be undone $\leftarrow$ **duplicated**

moreover:

- if $m.ckpt_num < RL_num_i$ then m is logged into current $cp_i.msg_log$ before being processed
- to allow replying in case of future rollback to $RL_num \geq m.ckpt_num$



On $recv_i(P_i, P_j, m)$ by C_i :

(Case 2) m was sent in the current incarnation ($m.inc_num = inc_num_i$)

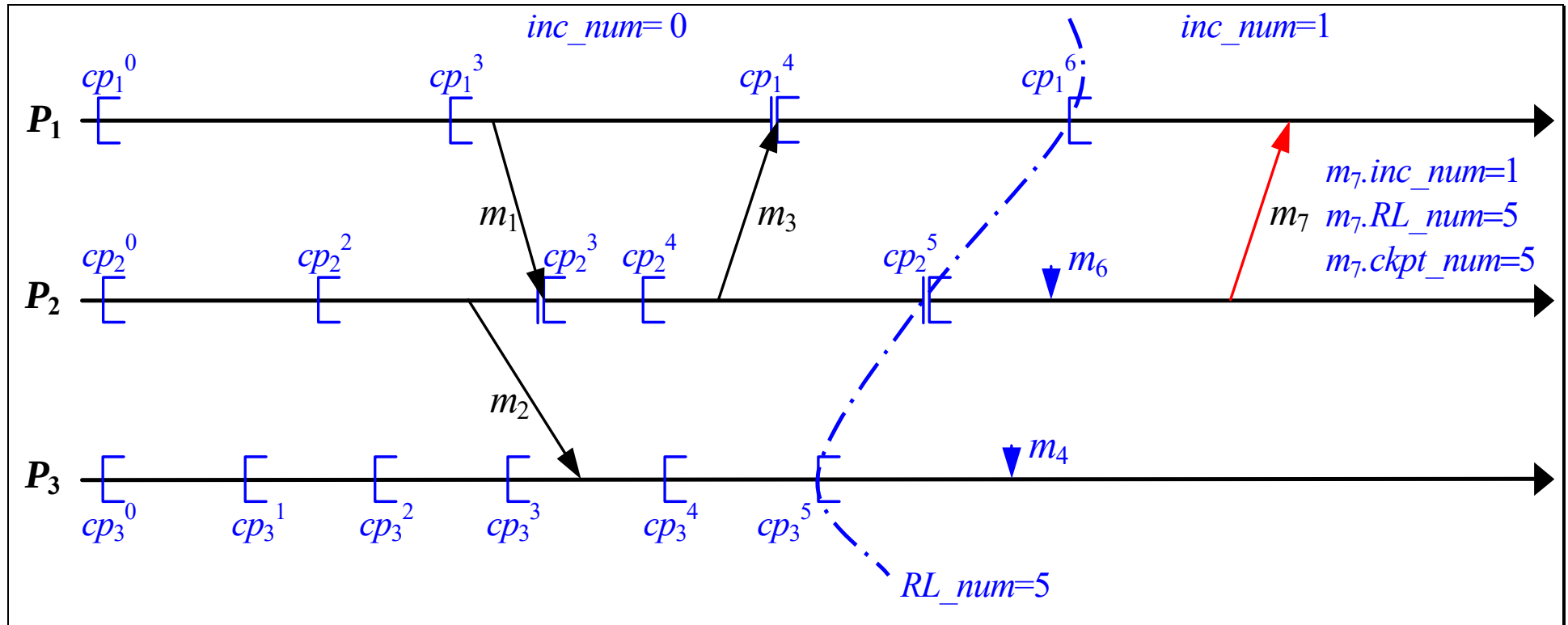
- if $m.ckpt_num < ckpt_num_i$ then m is logged before processing
- if C_i will further roll back P_i to the current checkpoint
 m will have to be replayed iff the new $RL_num > m.ckpt_num$
- if $m.ckpt_num > ckpt_num_i$ then m is processed after C_i takes
a new $cp_i :: cp_i.ckpt_num := m.ckpt_num$ and m is not logged
- if $m.ckpt_num = ckpt_num_i$ then m is processed without logging
or taking a new checkpoint

Message logging rule (following from Case 1 and Case 2)

m received by C_i is logged before delivery to P_i iff

$(m.inc_num < inc_num_i \wedge m.ckpt_num < RL_num_i)$ or

$(m.inc_num = inc_num_i \wedge m.ckpt_num < ckpt_num_i)$



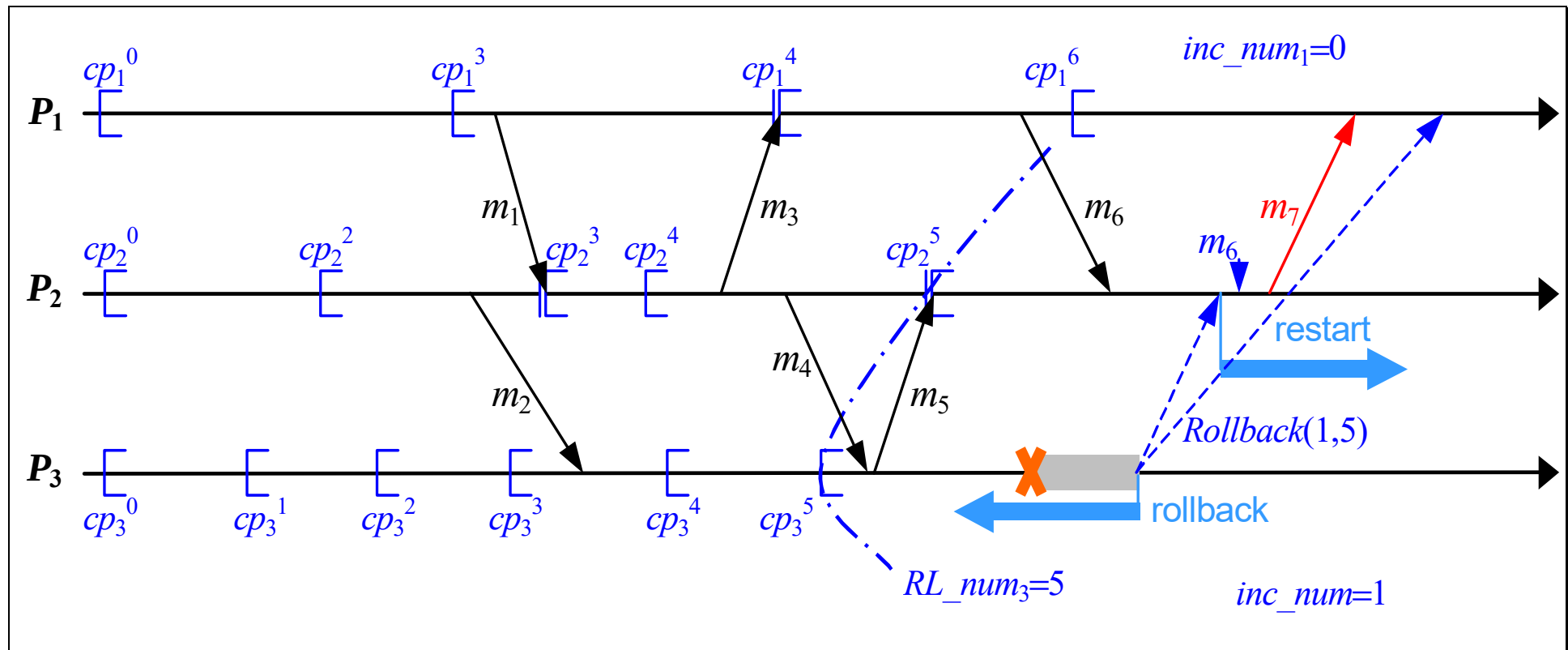
- $m_7.inc_num=inc_num_1=1$ and $m_7.ckpt_num=5 < ckpt_num_1=6$
- m_7 has to be logged before being processed because if P_1 will roll back to cp_1^6 ...
- ... then C_1 will have to replay $recv\ m_7$ if there will be $RL_num > 5$ (for that new recovery)



On $recv_i(\mathbf{P}_i, \mathbf{P}_j, \mathbf{m})$ by C_i :

(Case 3) $\mathbf{m}$ was sent in a future incarnation ($\mathbf{m}.inc_num > inc_num_i$)

- C_i comes to know of a recovery
- so it sets $RL_num_i := \mathbf{m}.RL_num$ and $inc_num_i := \mathbf{m}.inc_num$
- and rolls back to the earliest checkpoint $cp_i :: cp_i.ckpt_num \geq \mathbf{m}.RL_num$
- after rolling back $\mathbf{m}$ is handled as in Case 2 (since $\mathbf{m}.inc_num = inc_num_i$)



- m_7 was received before $Rollback(1,5)$ from C_3
- so on reception $m_7.inc_num=1$ but $inc_num_1=0$



QS algorithm (Manivannan-Singhal) recap:

- failed process rolls back to its latest checkpoint and inform others about that
- there is always a CP^* which contains this checkpoint = no domino-effect
- process can resume computation without waiting for other processes to rollback
= checkpointing and recovery are fully asynchronous
- communication-induced checkpointing intelligently guides the checkpointing to eliminate “useless” checkpoints (every CP is CP^*)
- only 3 integers are piggybacked with each message
- messages are pessimistically but selectively logged at the receiver
- garbage collection: after a process has established a RL, all local checkpoints preceding the line can be deleted

BIBLIOGRAPHY

- [1]  E.N. Elnozahy, W. Zwaenepoel, “Manetho, Transparent Rollback-Recovery with Low Overhead, Limited Rollback and Fast Output Commit”, IEEE Transactions on Computers, vol.41, no.5, pp.526-531, 1992.
- [2]  D. Manivannan, M. Singhal, “Quasi-synchronous checkpointing: Models, characterization, and classification”, IEEE Transactions on Parallel and Distributed Systems, pp. 703-713, 1996.
- [3]  A. D. Kshemkalyani, M. Singhal, “Distributed Computing – Principles, Algorithms, and Systems”, Cambridge University Press, 2008, ch.13.